

Haskell と圏論：

パフォーマンス改善から Kan 拡張へ

グミ@GummyCandy1206

関数型まつり 2026 2026-07-11

#fp_matsuri_b

自己紹介

グミ @GummyCandy1206

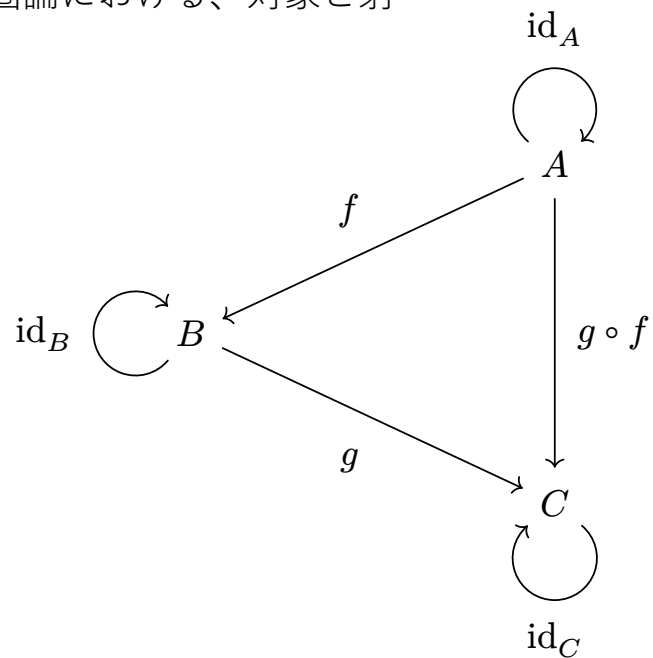
- 富山県のエンジニア
- 好きな言語: Haskell, Lean
- 趣味で関数型と圏論を勉強しています。
- 仕事でも関数型使いたい！

最近は圏論の Kan 拡張と Haskell の関係について勉強していました。この発表ではそのことについて話します。

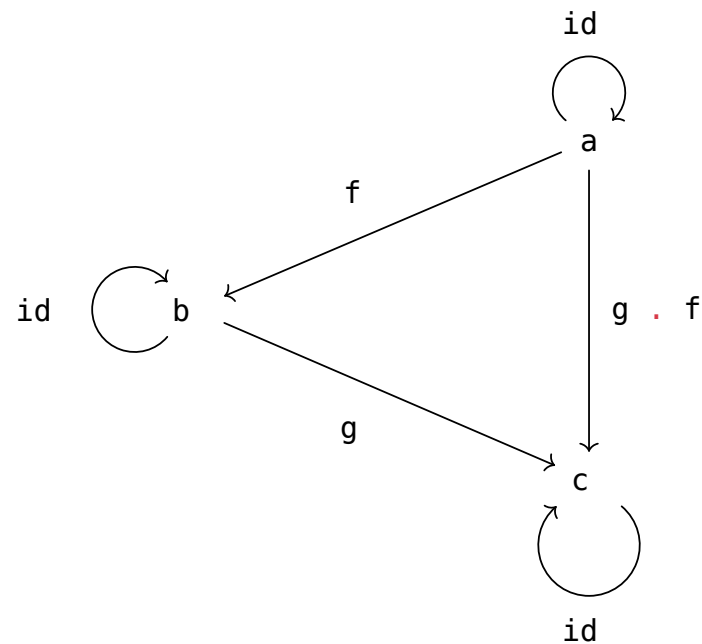
Haskell は圏論に由来するいくつかの概念を取り入れている。

例えば…

圏論における、対象と射

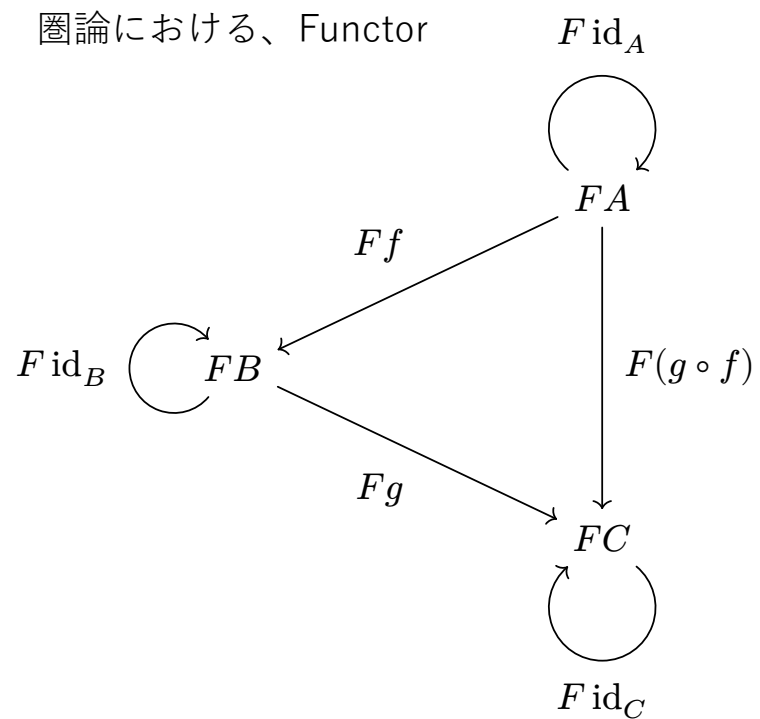


Haskell における、型と関数

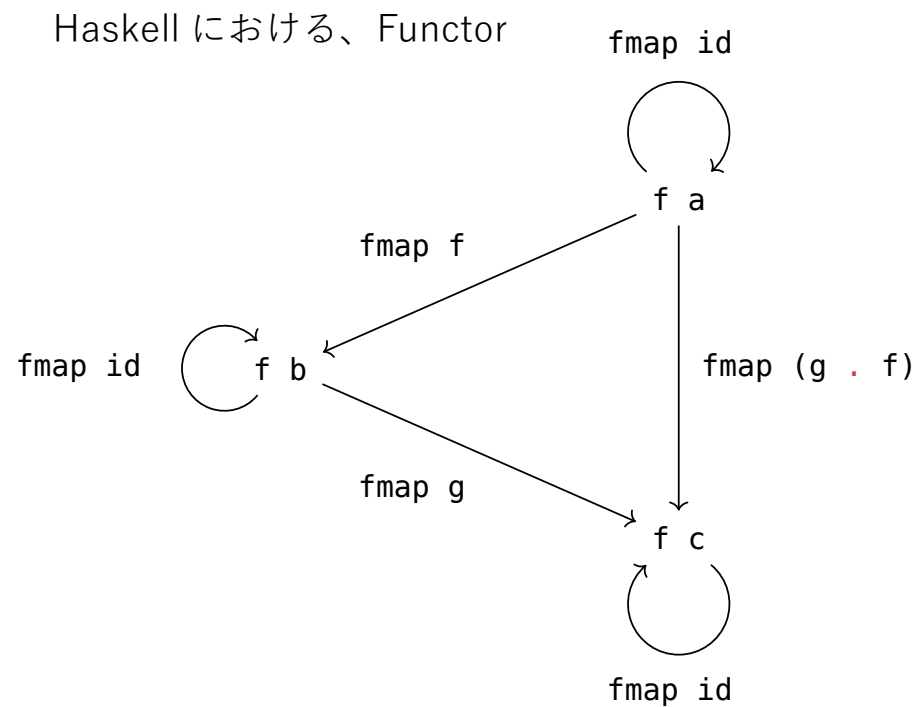


もちろん、それだけではなく...

圏論における、Functor



Haskell における、Functor



他にも…

圏論	Haskell
自然変換 $\alpha : F \Rightarrow G$	多相関数 forall a. f a -> g a
モナド (T, η, μ)	Monad m

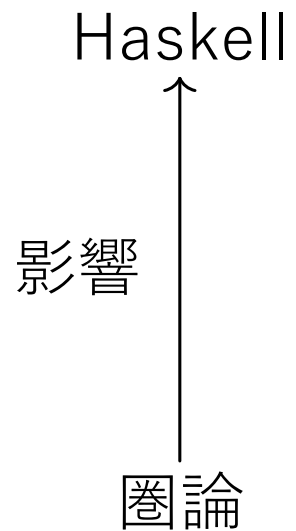
一方で、圏論には次の有名な言葉がある。

We end with the observations that **all concepts of category theory are Kan extensions.**

— Mac Lane, Saunders. Categories for the Working Mathematician.

圏論における**すべての概念は Kan 拡張！**

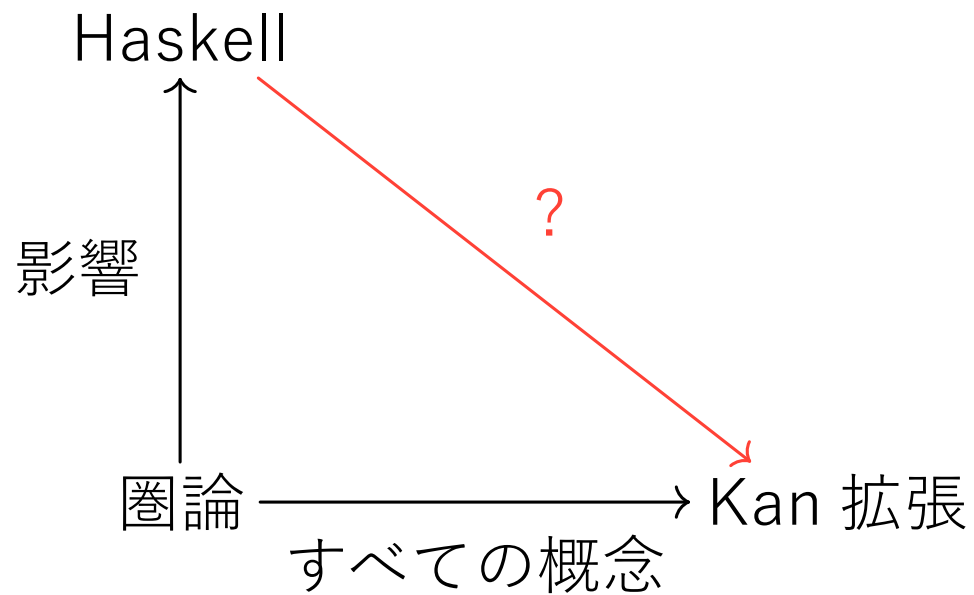
圏論は Haskell に影響を与えている。



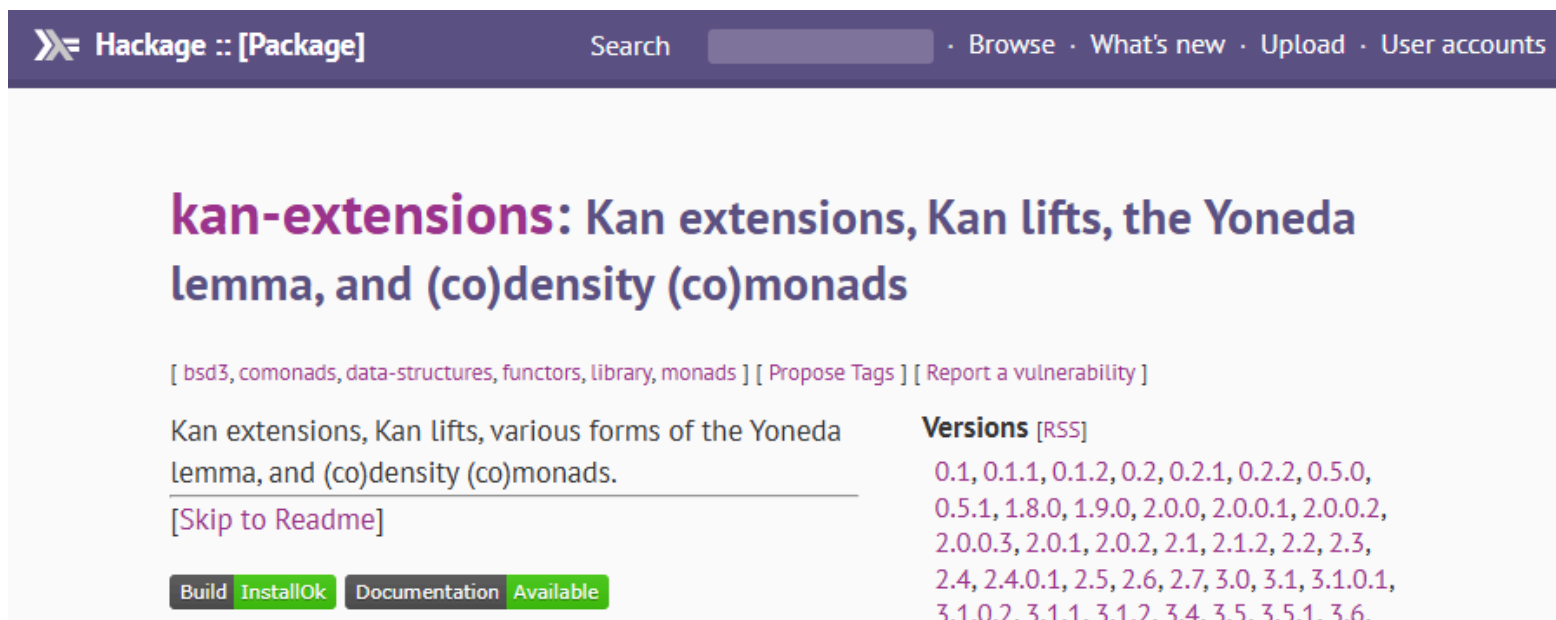
一方で、圏論におけるすべての概念は Kan 拡張である。



では、Haskell の概念も Kan 拡張として理解できるのでは？



“Haskell Kan extension”で調べてみると、なんと Kan 拡張に関する Haskell のパッケージが見つかる！



The screenshot shows the Hackage website interface. At the top is a dark purple header with the Hackage logo, a search bar, and navigation links: 'Browse', 'What's new', 'Upload', and 'User accounts'. The main content area has a light gray background. The package name 'kan-extensions' is prominently displayed in a large, bold, purple font, followed by a description: 'Kan extensions, Kan lifts, the Yoneda lemma, and (co)density (co)monads'. Below this, there are tags in a smaller purple font: '[bsd3, comonads, data-structures, functors, library, monads]', '[Propose Tags]', and '[Report a vulnerability]'. A short description follows: 'Kan extensions, Kan lifts, various forms of the Yoneda lemma, and (co)density (co)monads.', with a link '[Skip to Readme]'. At the bottom left, there are two buttons: 'Build' (gray) and 'InstallOk' (green), followed by 'Documentation' (gray) and 'Available' (green). On the right side, under the heading 'Versions [RSS]', a list of version numbers is shown in purple: '0.1, 0.1.1, 0.1.2, 0.2, 0.2.1, 0.2.2, 0.5.0, 0.5.1, 1.8.0, 1.9.0, 2.0.0, 2.0.0.1, 2.0.0.2, 2.0.0.3, 2.0.1, 2.0.2, 2.1, 2.1.2, 2.2, 2.3, 2.4, 2.4.0.1, 2.5, 2.6, 2.7, 3.0, 3.1, 3.1.0.1, 3.1.0.2, 3.1.1, 3.1.2, 3.4, 3.5, 3.5.1, 3.6,'.

参考：https://hackage.haskell.org/package/kan-extensions

この発表のゴール

kan-extensions パッケージのコードを通じて、
Haskell と Kan 拡張の関係を理解する。

Data.Functor.Kan.Ran に右 Kan 拡張が定義されている。

```
1 newtype Ran g h a
2   = Ran { runRan :: forall b. (a -> g b) -> h b }
```

```
1 newtype Ran g h a
2   = Ran { runRan :: forall b. (a -> g b) -> h b }
```

一見ただけでは

- どのように利用できるのかがわからない
- 圏論の右 Kan 拡張との関係が分からない

kan-extensions の中でも簡単で具体的な例から徐々に一般化して Haskell と Kan 拡張の関係を観察する。

Yoneda

次のようなコードを考える。

```
1 import Debug.Trace
2
3 data Box a = Box [a] deriving Show
4
5 instance Functor Box where
6   fmap f (Box xs) = trace "重い処理" $ Box (fmap f xs)
7
8   g :: Int -> Int
9   g x = x + 1
10
11 plain :: Box Int
12 plain = fmap g $
13         fmap g $
14         fmap g $
15         Box [1,2]
```

つまり、fmap が重い場合を考える。

(Box は説明用に定義した Functor)

このコードの実行結果は以下の通り。

```
1 ghci> plain
2 重い処理
3 重い処理
4 重い処理
5 Box [4,5]
```

重い処理が 3 回呼び出されている。

先程のコードの `plain` を次のように修正する。

```
1 import Data.Functor.Yoneda
2
3 yoneda :: Box Int
4 yoneda = lowerYoneda $
5     fmap g $
6     fmap g $
7     fmap g $
8     liftYoneda $
9     Box [1,2]
```

`kan-extensions` の `Data.Functor.Yoneda` で定義された関数を利用する。

修正は 2 箇所だけ。

```
1 plain :: Box Int
2 plain = fmap g $
3       fmap g $
4       fmap g $
5       Box [1,2]
6
7 yoneda :: Box Int
8 yoneda = lowerYoneda $
9       fmap g $
10      fmap g $
11      fmap g $
12      liftYoneda $
13      Box [1,2]
```

liftYoneda と lowerYoneda が追加されている。

修正後のコードの実行結果は以下の通り。

```
1 ghci> yoneda  
2 重い処理  
3 Box [4,5]
```

重い処理の呼び出しが 1 回だけになった。

一方で、出力結果に変化はない。

計算結果を変えずに高速化できた！

高速化の謎を探るため、`liftYoneda` と `lowerYoneda` の実装を調べる。

```
1 liftYoneda :: Functor f => f a -> Yoneda f a
2 liftYoneda a = Yoneda (\f -> fmap f a)
3
4 lowerYoneda :: Yoneda f a -> f a
5 lowerYoneda (Yoneda f) = f id
```

参考：<https://hackage-content.haskell.org/package/kan-extensions-5.2.8/docs/src/Data.Functor.Yoneda.html>

図で書くと...

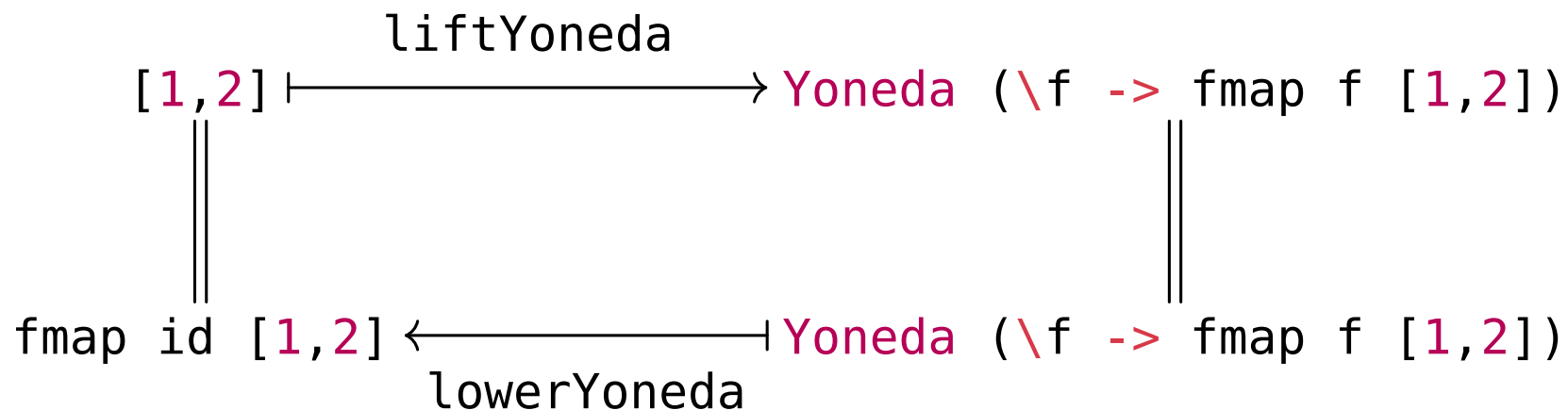
$$f\ a \begin{array}{c} \xrightarrow{\text{liftYoneda}} \\ \xleftarrow{\text{lowerYoneda}} \end{array} \text{Yoneda } f\ a$$

$f\ a == [Int]$ の場合は...

$$[Int] \begin{array}{c} \xrightarrow{\text{liftYoneda}} \\ \xleftarrow{\text{lowerYoneda}} \end{array} \text{Yoneda } []\ Int$$

具体例

[Int] の例として、[1,2] を考える。

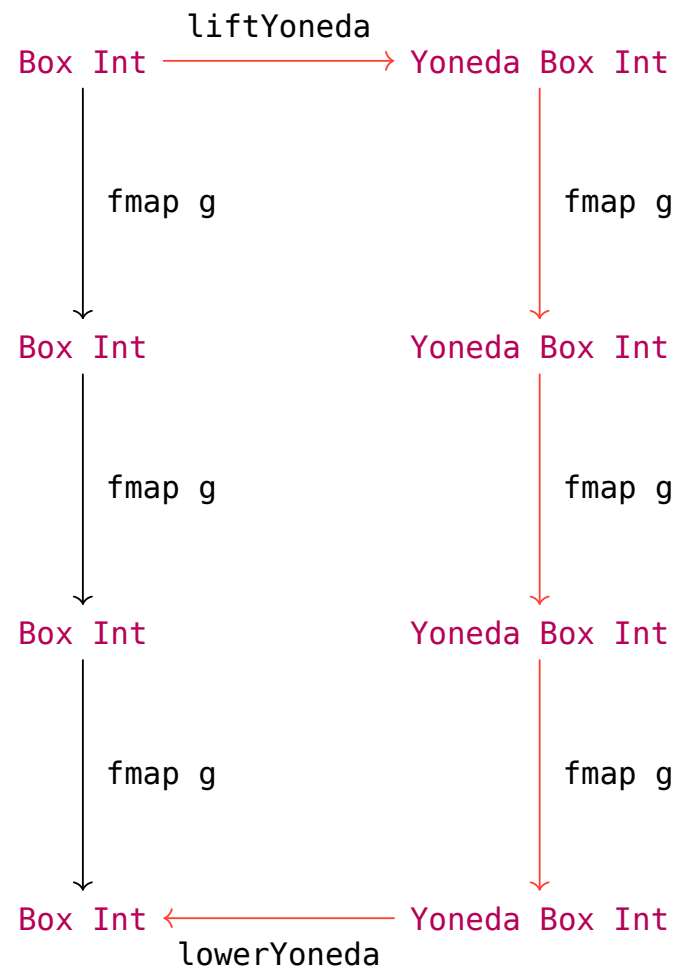


`liftYoneda` と `lowerYoneda` は互いに逆関数になっている。

yoneda は右図の赤い経路で計算している。

```
1 yoneda :: Box Int
2 yoneda = lowerYoneda $
3     fmap g $
4     fmap g $
5     fmap g $
6     liftYoneda $
7     Box [1,2]
```

高速化の理由を調べるため、Yoneda f の fmap の定義を確認する。



Yoneda f の fmap の定義

```
1 instance Functor (Yoneda f) where
2   fmap f m = Yoneda (\k -> runYoneda m (k . f))
```

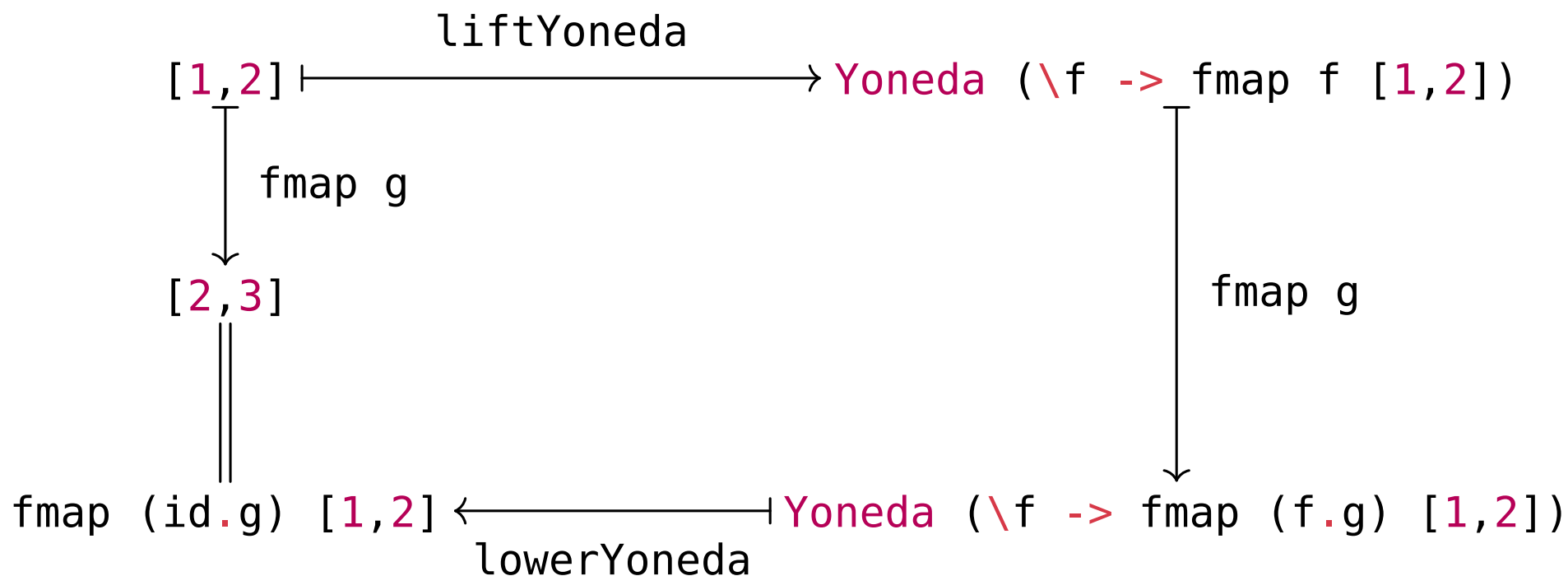
Yoneda の定義

```
1 newtype Yoneda f a
2   = Yoneda { runYoneda :: forall b. (a -> b) -> f b }
```

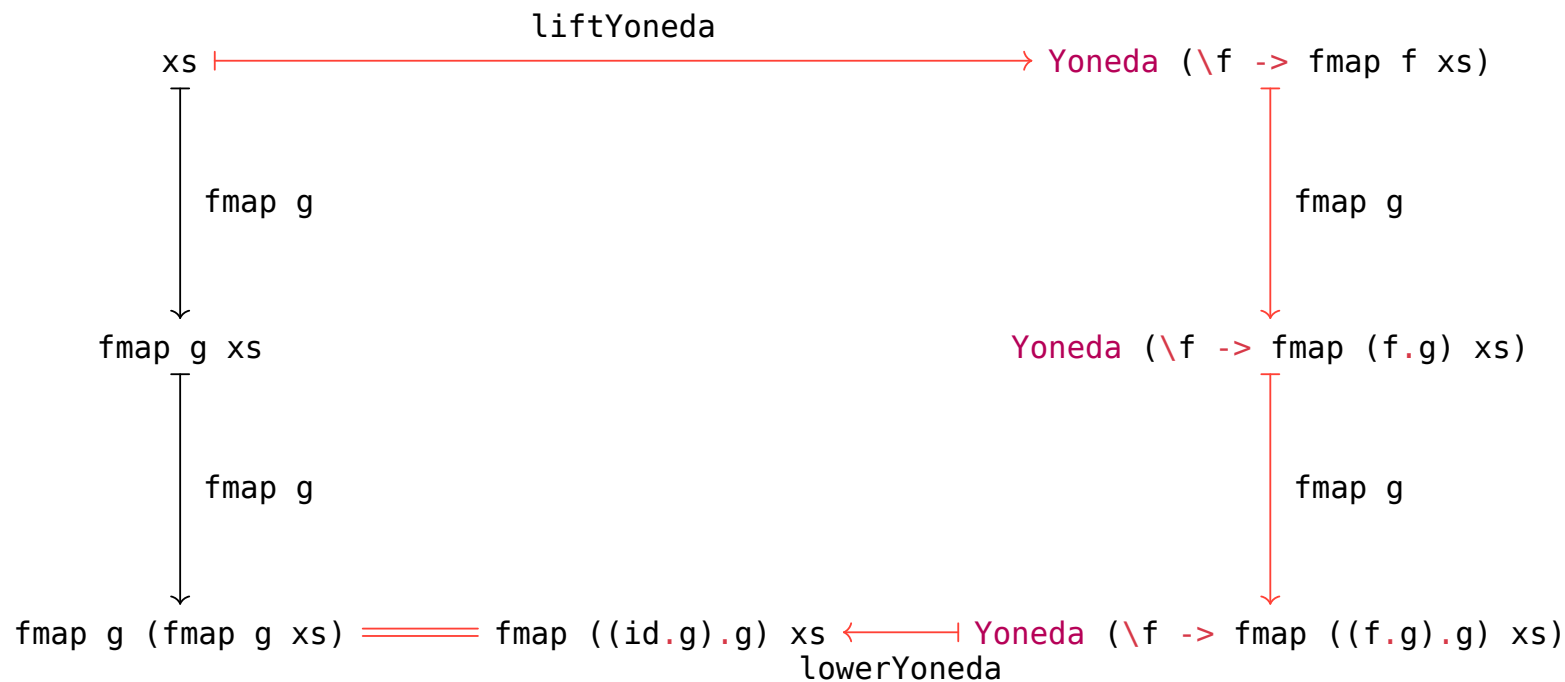
参考 : <https://hackage-content.haskell.org/package/kan-extensions-5.2.8/docs/src/Data.Functor.Yoneda.html>

具体例

$g\ x = x + 1$ として考える。



fmap g を繰り返す場合



fmap が満たすべき性質 $(\text{fmap } f) \cdot (\text{fmap } g) == \text{fmap } (f \cdot g)$

パフォーマンス改善の理由

Yoneda を経由することで、`fmap g` の合成が、`g` の合成の `fmap` に変わった。重い `fmap g` の処理が 1 回だけになり、パフォーマンスが改善した。

`(fmap g) . (fmap g) . (fmap g)`

改善



`fmap (g . g . g)`

注意

実際には **Yoneda** を用いたからといって直ちに `fmap` の合成のパフォーマンスが改善するわけではない。

GHC は `fmap` の合成を自動で最適化することがあるため、工夫しなくても十分に速い場合がある。

ここまでのまとめ

- Haskell と Kan 拡張の関係を知りたい
- Kan 拡張に関するパッケージ kan-extensions がある
- kan-extensions パッケージの使い道を知りたい
- 使い道の一つとして、**Yoneda** を用いた fmap の合成を最適化する例がある
- **Yoneda** は fmap の合成を合成の fmap に変換することで fmap の回数を 1 回にまとめることができる

Codensity

謝罪

プロポーザルでは **Codensity** でパフォーマンス改善する具体例を紹介すると書いていましたが、時間が足りないことと数式を並べるよりも良い説明を思いつかないことから割愛いたします。

この発表では具体例ではなく概要だけを紹介します。

ここまで、fmap の合成の最適化について扱った。

↓

fmap 以外はどうか？

Codensity を用いると、次のような例について同様の最適化ができる。

```
((m >>= k1) >>= k2) >>= k3
```


Codensity は **Yoneda** と似たような感じで定義されている。

```
1 newtype Yoneda f a = Yoneda
2   { runYoneda :: forall b. (a -> b) -> f b }
3
4 newtype Codensity m a = Codensity
5   { runCodensity :: forall b. (a -> m b) -> m b }
```

参考：<https://hackage-content.haskell.org/package/kan-extensions-5.2.8/docs/src/Control.Monad.Codensity.html>

fmap の合成は **Yoneda** を経由することで合成の fmap に変換できた。

$$(fmap\ g) \cdot (fmap\ f) \xrightarrow{\text{変換}} fmap\ (g \cdot f)$$

(>>=) の合成 $(m\ >>= k)\ >>= h$ は **Codensity** を経由することで次の形に変換できる。

$$(m\ >>= k)\ >>= h \xrightarrow{\text{変換}} m\ >>= (\backslash x \rightarrow k\ x\ >>= h)$$

(>>=) が満たすべき性質 $(m\ >>= k)\ >>= h \equiv m\ >>= (\backslash x \rightarrow k\ x\ >>= h)$

Codensity を経由することで、($\gg=$) の計算方法を変えることができる。 $m \gg= k$ の処理が重い場合、これを回避することでパフォーマンスを改善することができる。

$$\begin{array}{c} (m \gg= k) \gg= h \\ \downarrow \text{改善} \\ m \gg= (\backslash x \rightarrow k \ x \gg= h) \end{array}$$

ここまでのまとめ

- **Yoneda** とよく似た **Codensity** が kan-extensions に存在する。
- `fmap` の合成は **Yoneda** で高速化できる
- `(>>=)` の合成は **Codensity** で高速化できる

Ran

Yoneda と Codensity はよく似ている。

```
1 newtype Yoneda    f a
2   = Yoneda    { runYoneda    :: forall b. (a ->    b) -> f b }
3
4 newtype Codensity m a
5   = Codensity { runCodensity :: forall b. (a -> m b) -> m b }
```

R を次のように定義すると、この 2 つを一般化できそう。

```
1 newtype Yoneda      f a
2   = Yoneda { runYoneda :: forall b. (a -> b) -> f b }
3
4 newtype Codensity   m a
5   = Codensity { runCodensity :: forall b. (a -> m b) -> m b }
6
7 newtype R           g h a
8   = R { run         :: forall b. (a -> g b) -> h b }
```

実はこの **R** が、kan-extensions で定義されている右 Kan 拡張 **Ran** g h a 。

```
1 newtype Ran g h a
2   = Ran { runRan :: forall b. (a -> g b) -> h b }
```

$\text{Yoneda } f \ a \cong \text{Ran Identity } f \ a$

$\text{Codensity } g \ a \cong \text{Ran } g \ g \ a$

Ran は **Yoneda** と **Codensity** の一般化になっている。

あとは、**Ran** が圏論の右 Kan 拡張に対応していることを確認すればよい。

次の順番で確認する。

- **Ran** $g \circ h \circ a$ を圏論的に表すと $\text{Hom}(\text{Hom}(a, G \rightarrow), H)$
- 右 Kan 拡張の随伴より、上記が $\text{Hom}(\text{Hom}(a, -), \text{Ran}_G H)$ と同型
- 米田の補題より、上記が $\text{Ran}_G H(a)$ と同型

Haskell と圏論の関係を見るために、Haskell の型、関数などを圏論の言葉で書き直す。

- 型 a は対象 a
- 関数 $a \rightarrow b$ は $\text{Hom}(a, b)$ の元。
- $f\ a$ は 関手 F で a を移したものの $F(a)$ に対応する。
- $\text{forall } a. f\ a \rightarrow g\ a$ は自然変換 $\text{Hom}(F, G)$ に対応する。

※厳密には違うと思うが、一旦これで進める。

Haskell の右 Kan 拡張の定義は以下のとおり。

```
1 newtype Ran g h a
2   = Ran { runRan :: forall b. (a -> g b) -> h b }
```

これを圏の言葉に書き換えると、次のようになる。

$$\mathrm{Hom} (\mathrm{Hom} (a, G -), H)$$

- $\text{Ran}_G h_a$ を圏論的に表すと $\text{Hom}(\text{Hom}(a, G-), H) \leftarrow \text{OK}$
- 右 Kan 拡張の随伴より、上記が $\text{Hom}(\text{Hom}(a, -), \text{Ran}_G H)$ と同型
- 米田の補題より、上記が $\text{Ran}_G H(a)$ と同型

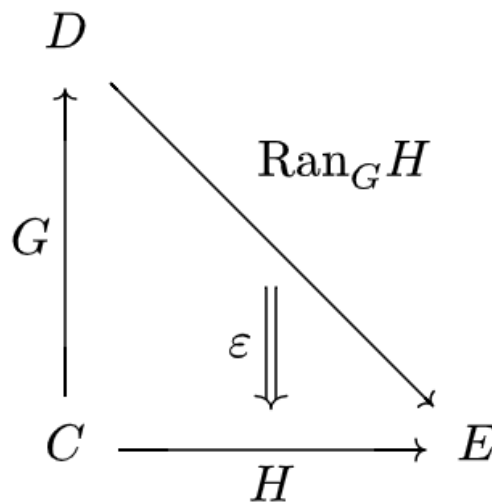
圏論における右 Kan 拡張の定義

C, D, E を圏として、 $G : C \rightarrow D, H : C \rightarrow E$ を関手とする。

$$\begin{array}{ccc} & D & \\ G \uparrow & & \\ C & \xrightarrow{H} & E \end{array}$$

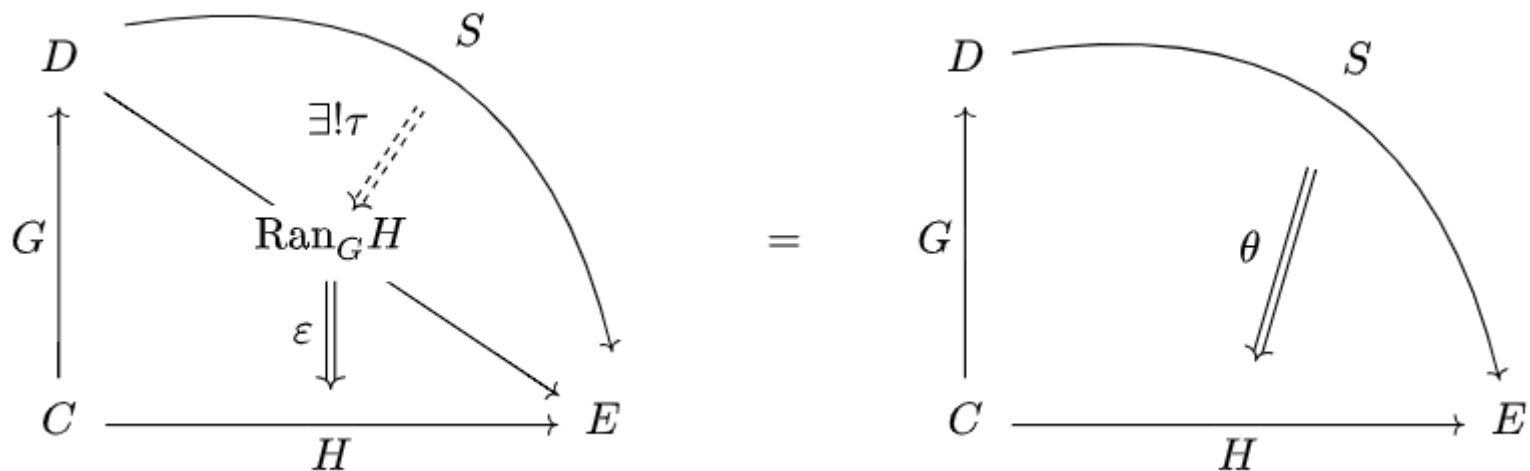
次の2つの条件を満たす $(\text{Ran}_G H, \varepsilon)$ を G に沿った H の右 Kan 拡張と呼ぶ。

1. $\text{Ran}_G H$ は関手 $D \rightarrow E$ で、 ε は自然変換 $\text{Ran}_G H \circ G \Rightarrow H$



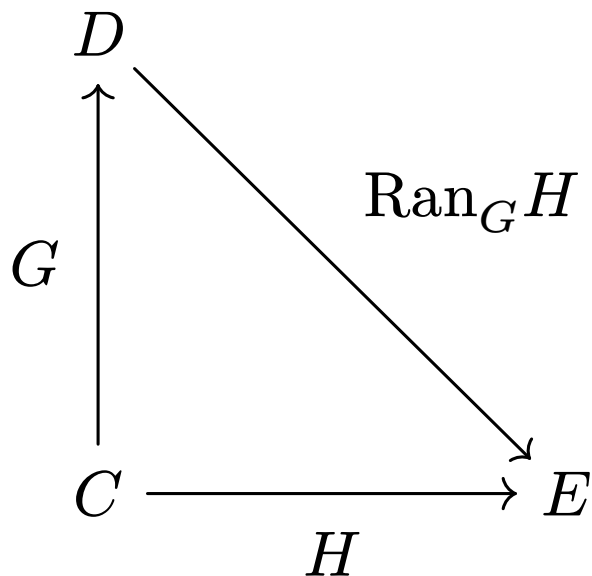
2. 関手 $S : D \rightarrow E$ と、自然変換 $\theta : S \circ G \Rightarrow H$ に対して、次を満たす自然変換 $\tau : S \Rightarrow \text{Ran}_G H$ が一意に存在する。

$$\theta = \varepsilon \circ \tau G$$



H を $\text{Ran}_G H$ に送る関手を Ran_G で表す。

関手 $C \rightarrow E$ からなる圏を E^C のように書くと、 Ran_G は関手 $E^C \rightarrow E^D$ 。



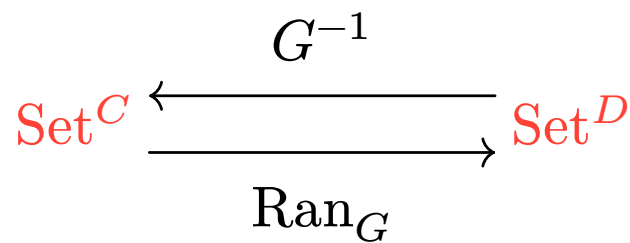
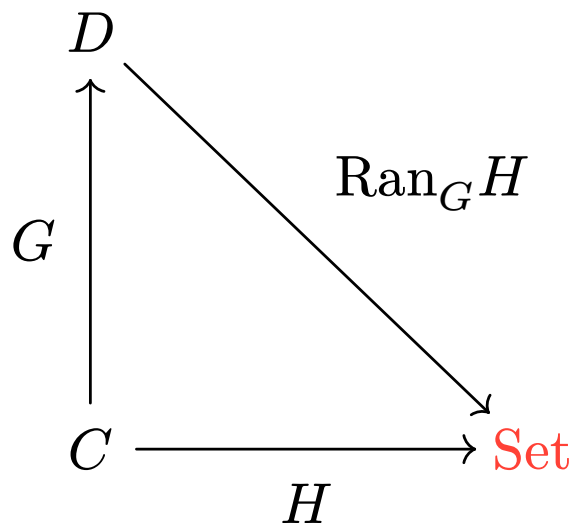
$$E^C \xrightarrow{\text{Ran}_G} E^D$$

関手 $\text{Ran}_G : E^C \rightarrow E^D$ には随伴関手（良い性質を持つ逆向きの関手）がある。 $G^{-1} : E^D \rightarrow E^C$ を $S \mapsto S \circ G$ とすると、 G^{-1} は Ran_G の左随伴になる。

$$\begin{array}{ccc}
 & G^{-1} & \\
 E^C & \xleftarrow{\quad} & E^D \\
 & \perp & \\
 & \xrightarrow{\quad} & \\
 & \text{Ran}_G &
 \end{array}$$

議論しやすいように、Haskell の圏 $\mathbf{Hask}$ の代わりに 集合の圏 $\mathbf{Set}$ で考える。

E を $\mathbf{Set}$ に置き換える。



随伴と $G^{-1} \dashv \text{Ran}_G$ と、随伴の定義より任意の $S \in E^D$ と $H \in E^C$ に対して、以下が成り立つ。

$$\text{Hom}_{\text{Set}^C}(G^{-1}S, H) \cong \text{Hom}_{\text{Set}^D}(S, \text{Ran}_G H)$$

S は任意なので、 $S = \text{Hom}_D(a, -)$ でも成り立つ。

$$\text{Hom}_{\text{Set}^C}(G^{-1}\text{Hom}_D(a, -), H) \cong \text{Hom}_{\text{Set}^D}(\text{Hom}_D(a, -), \text{Ran}_G H)$$

※任意の $b \in D$ に対して、 $\text{Hom}_D(a, b)$ は集合であると仮定する。つまり、 D は局所小圏とする。

$G^{-1} : E^D \rightarrow E^C$ は $S \mapsto S \circ G$ なので、次のように書き換えられる。

$$\mathrm{Hom}_{\mathrm{Set}^C}(G^{-1}\mathrm{Hom}_D(a, -), H) = \mathrm{Hom}_{\mathrm{Set}^C}(\mathrm{Hom}_D(a, G-), H)$$

まとめると

$$\begin{aligned} & \mathrm{Hom}_{\mathrm{Set}^C}(\mathrm{Hom}_D(a, G-), H) \\ &= \mathrm{Hom}_{\mathrm{Set}^C}(G^{-1}\mathrm{Hom}_D(a, -), H) \quad (G^{-1} \text{に書き換え}) \\ &\cong \mathrm{Hom}_{\mathrm{Set}^D}(\mathrm{Hom}_D(a, -), \mathrm{Ran}_G H) \quad (G^{-1} \dashv \mathrm{Ran}_G) \end{aligned}$$

- $\text{Ran } g \circ h \circ a$ を圏論的に表すと $\text{Hom}(\text{Hom}(a, G -), H) \leftarrow \text{OK}$
- 右 Kan 拡張の随伴より、上記が $\text{Hom}(\text{Hom}(a, -), \text{Ran}_G H)$ と同型 $\leftarrow \text{OK}$
- 米田の補題より、上記が $\text{Ran}_G H(a)$ と同型

示すべきこと

$$\mathrm{Hom}_{\mathrm{Set}^D}(\mathrm{Hom}_D(a, -), \mathrm{Ran}_G H) \cong \mathrm{Ran}_G H(a)$$

これは米田の補題より成り立つ。

米田の補題

D を局所小圏、 $F : D \rightarrow \mathrm{Set}$ を関手とする。このとき次が成り立つ。

$$\mathrm{Hom}_{\mathrm{Set}^D}(\mathrm{Hom}_D(a, -), F) \cong F(a)$$

Yoneda と米田の補題の関係

`Yoneda f a = Yoneda { runYoneda :: forall b. (a -> b) -> f b }`

$$\begin{array}{ccc} & \text{liftYoneda} & \\ f\ a & \xrightarrow{\hspace{1.5cm}} & \text{Yoneda } f\ a \\ & \xleftarrow{\hspace{1.5cm}} \text{lowerYoneda} & \end{array}$$

$$F(a) \xrightleftharpoons{\hspace{1.5cm}} \text{Hom}_{\text{Set}^D}(\text{Hom}_D(a, -), F)$$

- $\text{Ran } g \text{ h } a$ を圏論的に表すと $\text{Hom}(\text{Hom}(a, G -), H) \leftarrow \text{OK}$
- 右 Kan 拡張の随伴より、上記が $\text{Hom}(\text{Hom}(a, -), \text{Ran}_G H)$ と同型 $\leftarrow \text{OK}$
- 米田の補題より、上記が $\text{Ran}_G H(a)$ と同型 $\leftarrow \text{OK}$

まとめ

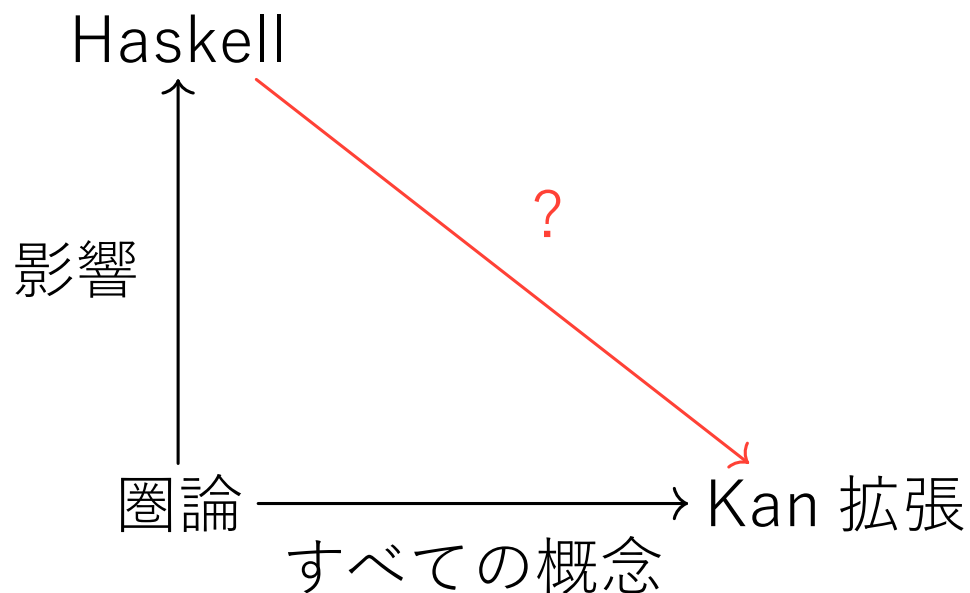
集合の圏 $\mathbf{Set}$ を Haskell の圏だと思えば、 $\mathbf{Ran} \, g \, h \, a$ と右 Kan 拡張 $\mathbf{Ran}_G H(a)$ が同型であることを確認できる。

$$\begin{aligned} & \mathbf{Hom}_{\mathbf{Set}^C}(\mathbf{Hom}_D(a, G-), H) && \text{(Haskell の右 Kan 拡張)} \\ \cong & \mathbf{Hom}_{\mathbf{Set}^D}(\mathbf{Hom}_D(a, -), \mathbf{Ran}_G H) && (G^{-1} \dashv \mathbf{Ran}_G) \\ \cong & \mathbf{Ran}_G H(a) && \text{(米田の補題)} \end{aligned}$$

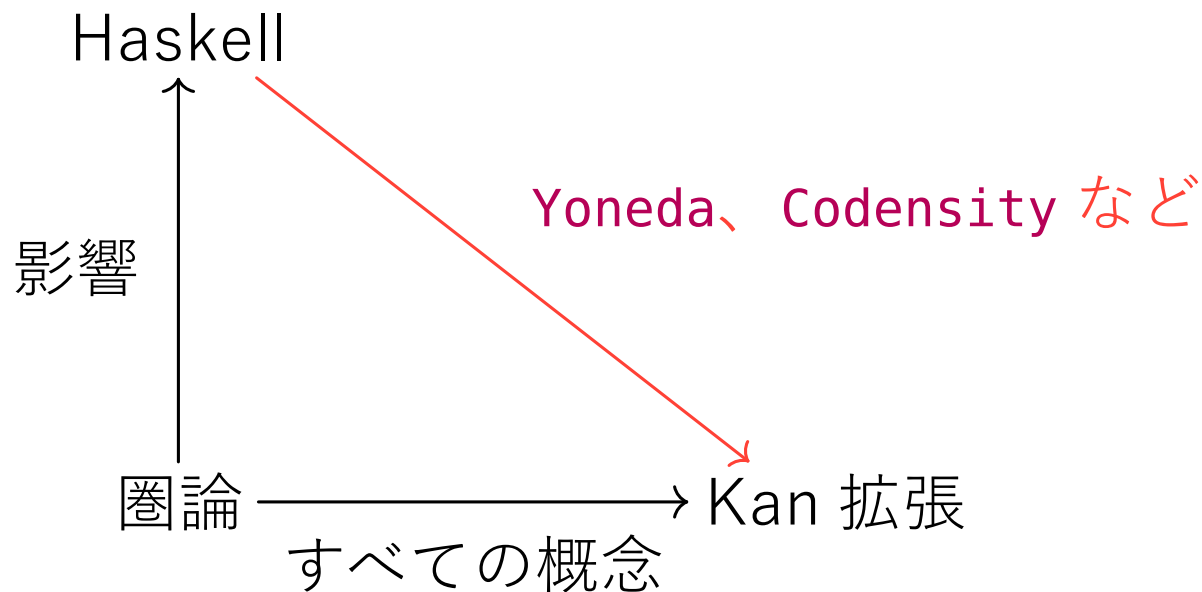
まとめ

最初の問い

Haskell の概念も Kan 拡張として理解できるのでは？



パフォーマンス改善に利用できる **Yoneda** や **Codensity** が Kan 拡張の例となっている。



まとめ

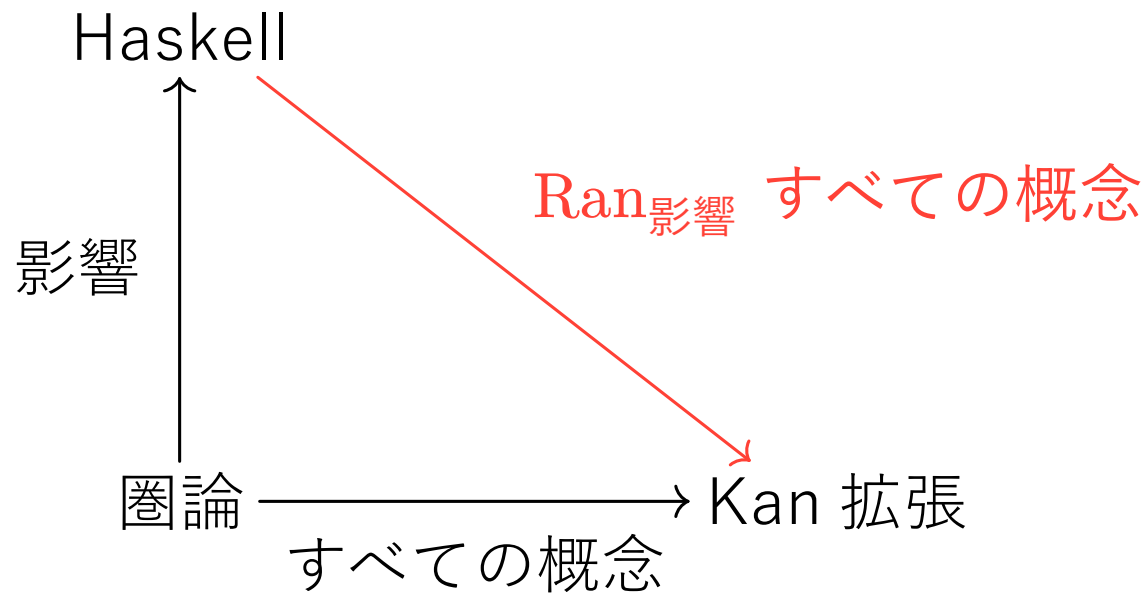
- **Yoneda** は `fmap` の合成のパフォーマンスを改善できる
- **Codensity** は `(>>=)` の合成のパフォーマンスを改善できる
- **Ran** は **Yoneda** と **Codensity** の一般化である
- **Ran** と右 Kan 拡張の対応は、右 Kan 拡張の随伴と米田の補題を用いて示せる

ご清聴ありがとうございました。

余談



この状況は Kan 拡張チャンス!!である。



「実はこの発表内容自体が Kan 拡張になっていたのだ」と言えると面白かったが、これは過言。

すべての概念は Kan 拡張

Kan 拡張の例

- 関手 $F : C \rightarrow D$ が右随伴を持つとき、その右随伴は $\text{Lan}_F \text{id}_C$
- 関手 $G : D \rightarrow C$ が左随伴を持つとき、その左随伴は $\text{Ran}_G \text{id}_D$
- 極限は右 Kan 拡張
- 余極限は左 Kan 拡張

随伴も極限も抽象的な概念。

随伴の例

- 自由群関手は忘却関手の左随伴
- 集合 A に対して $- \times A$ は $(-)^A$ の左随伴

極限の例

- 終対象
- 直積
- 等化子
- 引き戻し
- 逆極限

終対象の例

- 一元集合
- 自明群（始対象でもある）
- 整数の整除関係の圏における 0
- Unit 型 $()$

直積の例

- 集合の直積
- 位相空間の直積
- 整数の整除関係の圏における最大公約数
- タプル (a, b)

逆極限の例

- p -進整数
- 副有限群

参考文献

- [1] alg-d. Kan 拡張. https://alg-d.com/math/kan_extension/kan_extension.pdf.
- [2] Bartosz Milewski. Kan Extensions | Bartosz Milewski's Programming Cafe. <https://bartoszmilewski.com/2017/04/17/kan-extensions/>.
- [3] Edward A. Kmett. Kan-extensions. <https://hackage.haskell.org/package/kan-extensions>.
- [4] Saunders Mac lane and Saunders MacLane. Categories for the working mathematician, Vol. 5. Springer, 1971.
- [5] Tom Leinster. ベーシック圏論, 丸善出版, 2017.